

1. Nom du trinôme

Distanciel

HABRAN Karl - SEBASTIAO Luna – NGNIDJIE Alex-Claudiel

2. Idee du projet en une phrase

Transformer des données météo multi-villes en indicateurs analytiques fiables afin d'aider à comprendre et anticiper les variations climatiques locales.

3. API choisie

les API Open-Meteo + Nominatim

Analyse météo multi-villes avec enrichissement géographique

Le choix des APIs Open-Meteo et Nominatim s'explique par leur complémentarité : Nominatim permet de transformer une ville en coordonnées géographiques exploitables, tandis qu'Open-Meteo fournit des données météorologiques détaillées à partir de ces coordonnées. Cette combinaison permet de construire une pipeline ELT cohérente, allant de la localisation à l'analyse météo multi-villes.

4. Donnees recuperees

Entités :

- villes géolocalisées
- données météo associées

Champs principaux :

- ville, departement , latitude, longitude
- température, humidité, précipitations, vent

Fréquence :

- géolocalisation à la demande
- météo quotidienne

5. Environnement et reproductibilité

Technologies utilisées :

- **Python 3.11**
- **PostgreSQL**
- **Dagster**
- **dbt**
- **Docker / Docker Compose**
- **Power BI** pour la visualisation

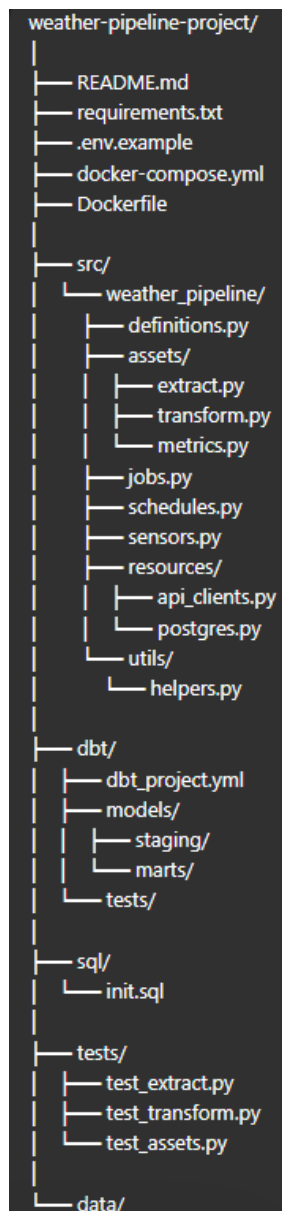
Le projet sera structuré autour de :

- un fichier `requirements.txt` pour les dépendances Python,
- un fichier `.env` pour les variables d'environnement,
- un fichier `docker-compose.yml` pour lancer l'ensemble des services nécessaires.

L'objectif est de garantir que l'ensemble du projet puisse être :

- installé facilement,
- relancé de manière identique sur différentes machines,
- exécuté dans un environnement stable et cohérent.

6. Structure du repo



7. Stockage retenu

Nous avons choisi **PostgreSQL** comme solution de stockage principale car il s'agit d'une base de données relationnelle robuste, largement utilisée dans les projets de data engineering et d'analytics.

Ce choix est pertinent pour notre projet car PostgreSQL permet :

- de stocker durablement les données météo collectées quotidiennement,
- de structurer clairement les tables brutes, intermédiaires et analytiques,
- de faciliter les transformations SQL avec **dbt**,

- de connecter facilement la base à **Power BI** pour la visualisation.

8. Pipeline vise

Extract :

- Récupération des villes à analyser (Python)
- Géolocalisation via **Nominatim**
- Récupération des données météo via **Open-Meteo**

Load :

- Insertion des données brutes dans la base

Transform :

- Nettoyage des colonnes
- Harmonisation des formats de date et de types

Objectif : Faire un Dashbord

9. Dagster

A remplir : Quels assets, quels jobs, quel schedule, quel sensor, quelles partitions si pertinentes

Assets :

Au niveau des assets, il y en a un pour les coordonnées des villes (Nom + Latitude/Longitude). Récupérer seulement la première fois, puisqu'à priori on ne risque pas de trouver de nouvelles villes.

Un autre asset pour récupérer les données météo brutes, et encore un autre pour les données météo nettoyées.

Un asset qui fait l'association de la localisation des villes et les données météo.

Pour finir, un asset pour les métriques des données météo, à savoir la moyenne de la température/précipitation/qualité de l'air, et leur minimum et maximum enregistré dans la journée et par ville.

Jobs :

Un job qui récupère les données météo brutes.

Un job qui va s'occuper de nettoyer, traiter et associer les différentes données météo.

Enfin, un job pour nettoyer les données trop vieilles.

Schedules :

Un schedule pour récupérer les données météo toutes les heures. Un autre tous les jours en heure creuse pour faire le calcul des données métriques météo. Un schedule pour nettoyer les données anciennes.

On peut aussi faire un snapshot sur les données météo brutes.

Sensors :

Un sensor pour les météo dangereuses ? Style orage, grêle. Le sensor déclencherait l'enregistrement dans une table à part qui répertorie les dates, lieux et le type de météo dangereuse, et qui ferait office de logs.

Partitions :

Une partition sur la date pour faciliter la récupération des données météo brutes.

10. dbt

A remplir : Quels modèles de staging, quel modèle analytique final, quels tests

11. Visualisation / aide à la décision

Outils : PowerBi

Exemples d'indicateurs

KPI

Alertes

- pluie élevée (> seuil)
- vent fort (> seuil)
- chute de température importante
- Qualité de l'air

Tendances

- Température en hausse ou baisse
- météo stable ou instable

Composants

Température moyenne par semaine

Ville la plus pluvieuse

Ville la plus ensoleillée

histogrammes de précipitations

courbes d'évolution de température

Carte de la France et de tous ses départements

Filtres

- Par département
- Par ville (commune)
- Par date

12. Tests et monitoring

Monitoring en 3 couches

Couche 1 — Monitoring pipeline : Dagster

Couche 2 — Monitoring qualité : dbt tests + SQL rules

Couche 3 — Monitoring d'audit : table PostgreSQL de suivi

13. Docker bonus

Oui Docker est quasi indispensable pour éviter les potentiels bugs en équipe.

Afin de garantir la reproductibilité, la portabilité et la simplicité de déploiement du projet

14. MVP fin de journée

Template Roadmap à finir

15. Repartition du travail

[HABRAN Karl](#)

PowerBI

PostgreSQL

[SEBASTIAO Luna](#)

Dagster

[NGNIDJIE Alex-Claudel](#)

DBT

[Travail commun](#)

Préparation de la soutenance

Readme

16. Risques connus

Nominatim : Nombre de requêtage limité à 1/s. Données crowdsourcées donc la qualité des données peut être affecté. Ne supporte pas des volumétries importantes.

Open-Météo : Temps de réponse variable. Agrégation de plusieurs modèles de données, donc possibilité de rencontrer des valeurs différentes pour la même position.

17. Plan B

Chercher une alternative pour les API si ça bloque.